PeakAware

Documentazione Tecnica

Ingegneria del Software · A.A. 2025/2026 · Gruppo 25

Fabio Tomasi · Damiano Bolech · Giacomo Brugnara

2026

Indice

1 Panoramica del Progetto **2**

2 Dipendenze **2**
2.1 Produzione 2
2.2 Sviluppo 2

3 Struttura delle Cartelle **2**

4 Endpoints API **3**
4.1 Autenticazione — `/api/auth` 3
4.2 Modulo Educazione — `/api/educazione` 3

5 Codici di Stato HTTP **4**

6 Autenticazione JWT **4**

7 Test Postman — Primo Commit **5**

1. Panoramica del Progetto

PeakAware è una piattaforma digitale (app + web) per avvicinare le persone alla montagna in modo consapevole, sicuro e sostenibile.
Il backend espone API REST consumate dal frontend Angular.

Stack tecnologico:

- Runtime: Node.js v24
- Framework HTTP: Express
- Database: MongoDB (Mongoose ODM)
- Autenticazione: JWT + bcryptjs
- Dev tool: nodemon

2. Dipendenze

2.1. Produzione

Pacchetto	Versione	Ruolo
`express`	^5.x	Framework HTTP: routing, middleware, REST
`mongoose`	^8.x	ODM MongoDB: modelli, schema, validazioni
`dotenv`	^16.x	Variabili d'ambiente da file `.env`
`bcryptjs`	^2.x	Cifratura password
`jsonwebtoken`	^9.x	Generazione e verifica token JWT
`cors`	^2.x	Richieste cross-origin da Angular (porta 4200)

2.2. Sviluppo

Pacchetto	Versione	Ruolo
`nodemon`	^3.x	Riavvio automatico del server ad ogni salvataggio

3. Struttura delle Cartelle

Percorso	Responsabilità
`backend/server.js`	Entry point: Express, middleware, routes
`backend/.env`	Variabili d'ambiente (non in Git)

`backend/src/config/database.js`	Connessione MongoDB tramite Mongoose
`backend/src/models/Utente.js`	Schema utente, cifratura password, verifica credenziali
`backend/src/models/Quiz.js`	Schema quiz con domande e risposte embedded
`backend/src/models/Video.js`	Schema video educativi e divulgativi
`backend/src/controllers/ /authController.js`	Logica registrazione e login
`backend/src/controllers/ /educazioneController.js`	CRUD quiz e video
`backend/src/routes/authRoutes.js`	Routing endpoint autenticazione
`backend/src/routes/ /educazioneRoutes.js`	Routing endpoint educazione
`backend/src/middleware/auth.js`	Verifica JWT e controllo ruolo admin

4. Endpoints API

4.1. Autenticazione — `/api/auth`

Metodo	URL	Protetto	Descrizione
POST	/api/auth/registrati	No	Crea account. Body: { email, password, eta }
POST	/api/auth/login	No	Autentica utente, restituisce token JWT

4.2. Modulo Educazione — `/api/educazione`

Metodo	URL	Protetto	Descrizione
GET	/api/educazione/quiz	No	Tutti i quiz. Filtri: ?categoria= &difficolta=
GET	/api/educazione/quiz/:id	No	Quiz completo con domande e risposte
POST	/api/educazione/quiz	Admin	Crea quiz (solo admin/-SAT)
PUT	/api/educazione/quiz/:id	Admin	Aggiorna quiz (solo admin/-SAT)
DELETE	/api/educazione/quiz/:id	Admin	Elimina quiz (solo admin/-SAT)
GET	/api/educazione/video	No	Tutti i video. Filtro: ?modulo=

POST	/api/educazione/video	Admin	Aggiunge video (solo admin/SAT)

5. Codici di Stato HTTP

Ogni risposta include sempre `successo` (boolean) e `messaggio` (stringa).
In caso di errore è presente anche `errore` con il dettaglio tecnico.

Codice	Nome	Quando viene restituito
200	OK	Operazione completata (GET, PUT, DELETE riusciti)
201	Created	Risorsa creata (POST /registrati, POST /quiz, POST /video)
400	Bad Request	Dati mancanti o non validi nel body
401	Unauthorized	Token assente, non valido o scaduto; credenziali errate
403	Forbidden	Utente autenticato ma senza permessi admin
404	Not Found	Risorsa non trovata (es. ID quiz inesistente)
500	Internal Server Error	Errore imprevisto o timeout database

Esempio risposta successo (201):

```
{ "successo": true, "messaggio": "Registrazione avvenuta con
   successo",
  "dati": { "id": "...", "email": "...", "ruolo": "utente", "
     token": "..." } }
```

Esempio risposta errore (401):

```
{ "successo": false, "messaggio": "Accesso negato: token
   mancante" }
```

6. Autenticazione JWT

Il sistema usa JSON Web Token per gestire le sessioni stateless.
Il token viene generato al login, inviato al frontend e allegato ad ogni richiesta successiva nell'header `Authorization: Bearer <token>`.
Il middleware `proteggi` lo verifica e aggiunge l'utente a `req.utente`.
Il token scade dopo 7 giorni.

7. Test Postman — Primo Commit

#	Metodo	URL	Risultato atteso	Esito
1	GET	/	Server attivo	**PASS**
2	POST	/api/auth/registrati	201 + token JWT	**PASS**
3	POST	/api/auth/login	200 + token JWT	**PASS**
4	POST	/api/educazione/quiz (no token)	401 token mancante	**PASS**
5	POST	/api/educazione/quiz (utente)	403 permessi insufficienti	**PASS**

Tutti i test superati. Il backend core è funzionante.